

HDOC DAY – 30

ALGORITHM:

1. Start.
2. Read the size n of the array.
3. Read n array elements.
4. Display the menu:
 - Bubble Sort and Binary Search
 - Insertion and Deletion
 - Right Rotation by k positions
 - Exit
5. Read the user's choice.
6. Use switch-case:
 - **Choice 1:** Sort the array using Bubble Sort, then ask for an element and search for it using Binary Search.
 - **Choice 2:** Ask whether the user wants insertion or deletion. Take a valid position and perform the operation.
 - **Choice 3:** Read k and rotate the array to the right by k positions.
 - **Choice 4:** Exit.
 - Otherwise, display "Invalid choice".
7. Repeat the menu until Exit is selected.
8. Stop.

C PROGRAM:

```
#include <stdio.h>
```

```
#define MAX 100
```

```
void display(int a[], int n);
```

```
void bubbleSort(int a[], int n);
```

```
void binarySearch(int a[], int n);  
void insertElement(int a[], int *n);  
void deleteElement(int a[], int *n);  
void rightRotate(int a[], int n);
```

```
int main()  
{  
    int a[MAX], n, i;  
    int choice, subChoice;  
  
    printf("Enter the number of elements: ");  
    scanf("%d", &n);  
  
    /* Validate array size */  
    while (n <= 0 || n > MAX)  
    {  
        printf("Invalid size. Enter size between 1 and %d: ", MAX);  
        scanf("%d", &n);  
    }  
  
    printf("Enter %d elements:\n", n);  
    for (i = 0; i < n; i++)  
    {  
        scanf("%d", &a[i]);  
    }  
  
    do  
    {
```

```
printf("\n----- MENU ----- \n");  
printf("1. Bubble Sort and Binary Search\n");  
printf("2. Insertion and Deletion\n");  
printf("3. Right Rotation by k positions\n");  
printf("4. Exit\n");
```

```
printf("Enter your choice: ");  
scanf("%d", &choice);
```

```
switch (choice)
```

```
{
```

```
    case 1:
```

```
        bubbleSort(a, n);
```

```
        printf("\nSorted array: ");
```

```
        display(a, n);
```

```
        binarySearch(a, n);
```

```
        break;
```

```
    case 2:
```

```
        printf("\n1. Insertion\n");
```

```
        printf("2. Deletion\n");
```

```
        printf("Enter your choice: ");
```

```
        scanf("%d", &subChoice);
```

```
        switch (subChoice)
```

```
{
```

case 1:

insertElement(a, &n);

break;

case 2:

deleteElement(a, &n);

break;

default:

printf("Invalid choice.\n");

}

break;

case 3:

rightRotate(a, n);

printf("Array after right rotation: ");

display(a, n);

break;

case 4:

printf("Exiting program...\n");

break;

default:

printf("Invalid menu choice.\n");

}

```
    } while (choice != 4);

    return 0;
}

/* Function to display the array */
void display(int a[], int n)
{
    int i;

    for (i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }

    printf("\n");
}

/* Function to perform Bubble Sort */
void bubbleSort(int a[], int n)
{
    int i, j, temp;

    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - 1 - i; j++)
```

```

{
    if (a[j] > a[j + 1])
    {
        temp = a[j];
        a[j] = a[j + 1];
        a[j + 1] = temp;
    }
}
}
}

```

/* Function to perform Binary Search */

void binarySearch(int a[], int n)

```

{
    int key;

    int low = 0, high = n - 1;

    int mid, found = 0;

    printf("Enter element to search: ");
    scanf("%d", &key);

    while (low <= high)
    {
        mid = (low + high) / 2;

        if (a[mid] == key)
        {

```

```

        printf("%d found at position %d.\n", key, mid + 1);

        found = 1;

        break;
    }

    else if (key < a[mid])
    {
        high = mid - 1;
    }

    else
    {
        low = mid + 1;
    }
}

if (found == 0)
{
    printf("%d not found in the array.\n", key);
}
}

```

```

/* Function to insert an element */
void insertElement(int a[], int *n)
{
    int element, position, i;

    if (*n == MAX)
    {

```

```
    printf("Array is full. Insertion is not possible.\n");
    return;
}

printf("Enter element to insert: ");
scanf("%d", &element);

printf("Enter position (1 to %d): ", *n + 1);
scanf("%d", &position);

if (position < 1 || position > *n + 1)
{
    printf("Invalid position.\n");
    return;
}

for (i = *n; i >= position; i--)
{
    a[i] = a[i - 1];
}

a[position - 1] = element;
(*n)++;

printf("Array after insertion: ");
display(a, *n);
}
```

```
/* Function to delete an element */  
void deleteElement(int a[], int *n)  
{  
    int position, i;  
  
    if (*n == 0)  
    {  
        printf("Array is empty. Deletion is not possible.\n");  
        return;  
    }  
  
    printf("Enter position to delete (1 to %d): ", *n);  
    scanf("%d", &position);  
  
    if (position < 1 || position > *n)  
    {  
        printf("Invalid position.\n");  
        return;  
    }  
  
    for (i = position - 1; i < *n - 1; i++)  
    {  
        a[i] = a[i + 1];  
    }  
  
    (*n)--;
```

```
    printf("Array after deletion: ");  
    display(a, *n);  
}
```

```
/* Function to rotate array to the right */
```

```
void rightRotate(int a[], int n)
```

```
{
```

```
    int k, i, j, last;
```

```
    printf("Enter the value of k: ");
```

```
    scanf("%d", &k);
```

```
    if (k < 0)
```

```
    {
```

```
        printf("Invalid value of k.\n");
```

```
        return;
```

```
    }
```

```
/* Handles k greater than array size */
```

```
k = k % n;
```

```
for (i = 0; i < k; i++)
```

```
{
```

```
    last = a[n - 1];
```

```
    for (j = n - 1; j > 0; j--)
```

```
    {
```

```

    a[j] = a[j - 1];
}

```

```

a[0] = last;
}
}

```

TEST CASE TABLE:

TEST	INPUT ARRAY	OPERATION	ADDITIONAL INPUT	EXPECTED RESULT
1	5 2 8 1 4	Bubble Sort	Search 8	Sorted: 1 2 4 5 8; 8 found
2	10 20 30 40	Insertion	Insert 25 at position 3	10 20 25 30 40
3	10 20 30 40 50	Deletion	Delete position 3	10 20 40 50
4	1 2 3 4 5	Right Rotate	k = 2	4 5 1 2 3
5	1 2 3 4 5	Right Rotate	k = 7	4 5 1 2 3
6	10 20 30	Insertion	Position 5	Invalid position
7	10 20 30	Deletion	Position 0	Invalid position

```

(kali㉿kali)-[~]
$ vi h30.c

(kali㉿kali)-[~]
$ gcc h30.c -o h30

(kali㉿kali)-[~]
$ ./h30
Enter the number of elements: 5
Enter 5 elements:
5 2 8 1 4

—— MENU ——
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
Enter your choice: 1

Sorted array: 1 2 4 5 8
Enter element to search: 8
8 found at position 5.

```